

Web ve Mobil Uygulamalarda Business Logic Zafiyetleri

Ahmet Eren Arslan

Giriş

Günümüzde web ve mobil uygulamalarda güvenlik yalnızca teknik açıklardan ibaret değildir. SQL Injection, XSS veya zayıf kimlik doğrulama gibi klasik zafiyetlerin yanında, uygulamanın iş akışından kaynaklanan mantık hataları da ciddi güvenlik riskleri oluşturabilir.

Business Logic Zafiyeti Nedir?

Business Logic zafiyeti, bir sistemin dizaynı ve implementasyonu sırasında saldırılara açık hale gelmesine neden olacak tasarım ve mantık hatalarıdır.

Bunu kendi gündelik hayatımdan bir örnekle şöyle anlatabilirim: Okulumuzun yemekhanelerinde ücretsiz yemek yeme hakkı kazandım. Bu yüzden de okul kartımı yemekhanede okuttuğum zaman ücretsiz geçiş yapabiliyorum. Okulumuzda üç adet yemekhane var. Normal şartlar altında, her öğrenci okulda yalnızca bir öğün tüketmeli. Ancak bana tanınan bu ücretsiz geçişi, okulumuzdaki üç farklı yemekhanede de kullanarak bir günde üç kez ücretsiz öğün tüketebiliyorum. Kötü bir senaryoda, birisi kartın bu zafiyetinden yararlanarak yemekhane ücreti ödeyen birisine, yemekhane ücretinden daha ucuza bu kartı kiralayabilir. Bu benim kendi hayatımda karşılaştığım güzel bir Business Logic zafiyeti örneğidir.

Web ve Mobil Uygulamalarda Business Logic Zafiyetleri

Web ve mobil uygulamalarda da aslında durum oldukça benzerdir.

Business Logic zafiyetleri; web ve mobil uygulamalarda e-ticarette sepette ödeme yapmadan sipariş oluşturma, fiyat manipülasyonu, tekrarlı veya uygun olmayan kupon kullanımı; bankacılıkta negatif gibi yanlış para tutarları; rezervasyon yapılabilen uygulamalarda kapasite kontrolünün yapılmaması vb. durumlar şeklinde karşımıza çıkabilen açıklardır.

En Sık Karşılaşılan Web ve Mobil Uygulama Business Logic Zafiyetleri

1. **Fiyat Manipülasyonu:** Saldırganın kullanıcı tarafı parametrelerle fiyat üzerinde değişiklik yapabilmesi açıktır.
2. **Şifre Yenileme Açıklarıyla Hesap Ele Geçirme:** Saldırganın, başka bir kullanıcının hesabına erişmesini sağlayabilen şifre yenileme metotlarını kötüye kullanabilmesi gibi durumlara neden olan açıklardır.
3. **Miktar Sınırından Yararlanma:** Kullanıcının satın alabileceği, kullanabileceği ya da erişebileceği öğe sayısı ile ilgili kısıtlamaları atlaması sonucu oluşan açıklardır.
4. **Kusurlu Rol Yönetimiyle Ayrıcalık Yükseltme:** Kullanıcı rollerine uygunsuz izinler verilmesi sonucu ortaya çıkan açıklardır.
5. **Çok Adımlı Akış Manipülasyonu:** Sipariş veya ödeme onayı gibi çok adımlı aşamaları atlamaya olanak sağlayabilen zafiyetlerdir.

Bunlar web ve mobil uygulamalarda en çok karşılaşılan Business Logic zafiyetleri arasında yer almaktadır.

Tarihte Gerçekleşmiş Web ve Mobil Uygulama Business Logic Zafiyetleri

1. **Stripe Sınırsız İndirim Kullanımı:** HackerOne üzerinden bir kullanıcı, Stripe’ta bir ücret indiriminin aynı hesapta birden fazla kez uygulanabileceğini keşfetti. (HackerOne)
2. **Starbucks Gift Card Race Condition Olayı:** Starbucks hediye kartlarının çıktığı dönemlerde, hediye kartları arasında bakiye transferi yapılırken aynı transfer işlemi eş zamanlı iki kez tetiklenebiliyordu. (SecurityWeek)
3. **Uber Ücretsiz Yolculuk Bug’ı:** Anand Prakash adında bir kullanıcı, Uber’de geçersiz ödeme yöntemi olarak abc veya xyz gibi değerler girildiğinde yolculuğun ücretlendirilmeden tamamlanabildiğini fark etti. (TechCrunch)

Örnek Uygulamalar

Fiyat Manipülasyonu

Normal bir sepete ürün ekleme isteği aşağıdaki gibi olabilir:

```
POST /api/cart/add HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "product_id": 1024,
  "quantity": 1,
  "price": 1500
}
```

Saldırgan bu isteği yakalayarak fiyat değerini değiştirebilir:

```
POST /api/cart/add HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "product_id": 1024,
  "quantity": 1,
  "price": 1
}
```

Eğer sunucu fiyat bilgisini kendi veritabanından doğrulamak yerine kullanıcıdan gelen değere güvenirse, ürün gerçek fiyatından çok daha düşük bir tutarla satın alınabilir.

2FA Logic Bypass

Bir giriş sürecinde Two-Factor Authentication yani 2FA kullanıldığını düşünelim. Normal akış aşağıdaki gibi ilerler:

1. Kullanıcı, kullanıcı adı ve parolasını girer.
2. Sunucu kimlik bilgilerini doğrular ve kullanıcıyı `/login/otp` sayfasına yönlendirir.
3. Kullanıcı OTP kodunu girer.
4. Sunucu OTP kodunu doğrular ve kullanıcıyı `/dashboard` sayfasına yönlendirir.

Bu akışta bir mantık hatası, sunucunun birinci adımdan hemen sonra kullanıcıya authenticated session cookie vermesi, ancak kullanıcının dashboard sayfasını görmesini yalnızca arayüz üzerinden engellemeye çalışması durumunda ortaya çıkabilir. Böyle bir senaryoda saldırgan, kullanıcı adı ve parola ile giriş yaptıktan sonra OTP adımını tamamlamadan manuel olarak `/dashboard` adresine gitmeyi deneyebilir. Eğer backend tarafında 2FA kontrolü yapılmıyorsa, saldırgan OTP gereksinimini tamamen atlatabilir.

Benzer bir durumda uygulama, 2FA durumunu session içinde boolean bir flag ile takip ediyor olabilir:

```
// Vulnerable server-side logic - Pseudocode
if (user.checkPassword(pass)) {
    session.user_id = user.id;
    session.needs_2fa = true;
    res.redirect('/otp-page');
}

// Dashboard check
if (session.user_id) {
    // Missing check for session.needs_2fa == false
    res.render('dashboard');
}
```

Bu örnekte geliştirici, dashboard route üzerinde `needs_2fa` değerinin `false` olup olmadığını kontrol etmeyi unutmuştur. Bu nedenle kullanıcı, yalnızca parola aşamasını geçerek oturum açmış gibi davranabilir. Bu tarz zafiyetlerde problem OTP mekanizmasının var olmaması değil, OTP kontrolünün kritik endpoint'lerde zorunlu tutulmamasıdır.

Infinite Discount Loop

Bazı uygulamalar, kullanıcıların birden fazla kupon kodu uygulamasına izin verir. Ancak sistem, toplam indirimin toplam fiyatı geçip geçmediğini kontrol etmiyorsa bir Business Logic zafiyeti oluşabilir.

Örneğin bir saldırgan, `WELCOME10` adında 10 dolarlık bir indirim kuponu bulmuş olsun. Eğer uygulama aynı kupon kodunun tekrar tekrar uygulanmasına izin veriyorsa veya farklı kuponların toplam fiyat sıfırın altına düşene kadar birikmesine izin veriyorsa, saldırgan ürünleri ücretsiz alabilir ya da uygulamanın indirim/bakiye mantığını kötüye kullanabilir.

Tekrarlı kupon isteği aşağıdaki gibi olabilir:

```
POST /cart/apply-coupon HTTP/1.1
Host: example-store.com

coupon_code=WELCOME10&coupon_code=WELCOME10&coupon_code=WELCOME10
```

Eğer backend tarafında hesaplama şu şekilde yapılıyorsa:

```
Total = Total - (CouponValue * Count)
```

ve `Count` değeri 1 ile sınırlandırılmıyorsa, uygulamanın indirim mantığı bozulmuş demektir. Bu durumda sistem, aynı kuponun birden fazla kez kullanılmasını engellemediği için toplam tutar hatalı hesaplanabilir.

Etki Analizi

Bu zafiyetler yazılımsal veya donanımsal açıkların aksine doğrudan şirket verilerini ve gelirlerini etkileyebilecek niteliktedirler.

- **Finansal Kayıp:** Doğrudan indirim, kupon kullanımı vb. yöntemlerle maddi hasarlar verebilirler.

- **Veri İhlalleri:** Yetkisiz erişim ile başka kullanıcı verilerine erişim sağlayabilirler.
- **İtibar Hasarı:** Şirketin itibarında ve güvenilirliğinde ciddi sarsıntılara yol açabilirler.
- **Operasyonel Kesinti:** Saldırganlar açıklardan yararlanmak adına diğer kullanıcıların erişimlerinde kesintilere yol açabilirler.

Önleme Yöntemleri

- Güvenlik açısından kritik olan değerler sunucu tarafında yeniden türetilmelidir. Fiyat, yetki, sahiplik ve işlem durumu gibi değerler için istemci tarafındaki veri yalnızca girdi olarak kabul edilmeli, doğruluğun kaynağı olarak görülmemelidir.
- İş akışları açık bir state machine mantığıyla uygulanmalıdır. Adımların sırası yalnızca kullanıcı arayüzüyle değil, backend tarafında da zorunlu hale getirilmelidir.
- Eş zamanlılık gerçek bir tehdit olarak değerlendirilmelidir. İki isteğin aynı anda çalışması mümkünse, saldırının bunu deneyeceği varsayılmalıdır.
- Rate-limit ve izleme kontrolleri yalnızca kimlik doğrulama seviyesinde değil, özellik bazında da uygulanmalıdır. Referans sistemi, kupon kullanımı ve şifre sıfırlama gibi kötüye kullanıma açık özellikler kendi kontrollerine sahip olmalıdır.
- Tehdit modellemesi yalnızca teknik mimari üzerinden değil, iş süreci üzerinden de yapılmalıdır. Sadece saldırının zararlı payload göndermesi değil, kullanıcının dürüst davranmadığı durumda sistemin nasıl tepki vereceği de değerlendirilmelidir.

Sonuç

Business Logic zafiyetleri, uygulamanın teknik olarak çalışmasına rağmen iş kurallarının yanlış uygulanması nedeniyle ortaya çıkar. Bu da sistemi dışarıdan gelebilecek saldırılara veya istenmeyen olaylara yol açabilecek zafiyetler yaratır. Güvenlik açıkları yalnızca donanımsal veya yazılımsal olarak ortaya çıkan bir husus değildir. Bu nedenle güvenlik testlerinde yalnızca otomatik araçlara değil, manuel analiz ve iş akışı testlerine de önem verilmelidir.

Kaynakça

- <https://portswigger.net/web-security/logic-flaws>
- <https://bitpanic.medium.com/top-10-overlooked-business-logic-flaws-in-web-applications>
- <https://owasp.org/www-project-top-10-for-business-logic-abuse/>
- <https://blogs.jsmon.sh/what-is-business-logic-vulnerability-ways-to-exploit-examples>
- https://cheatsheetseries.owasp.org/cheatsheets/Business_Logic_Security_Cheat_Sheet.html